



Funciones Definidas por el Usuario en SQL

En bases de datos relacionales como PostgreSQL, las funciones definidas por el usuario (UDF, User Defined Functions) permiten encapsular lógica en bloques reutilizables que extienden el lenguaje SQL.

¿Qué son las Funciones Definidas por el Usuario?

En lugar de repetir consultas complejas, un analista o desarrollador puede escribir una función que reciba parámetros de entrada, procese información y devuelva un valor o conjunto de filas.

Una UDF en SQL es comparable a una función en cualquier lenguaje de programación (Python, C, Java), pero su ámbito se centra en manipular y consultar datos. Al ser ejecutadas dentro del motor de la base de datos, permiten:

Centralizar lógica

Evitando duplicación en múltiples consultas

Optimizar rendimiento

Se ejecutan en el servidor y reducen tráfico de red

Mejorar seguridad

Ocultando detalles de tablas y delegando acceso controlado

Tipos de Funciones en PostgreSQL

PostgreSQL soporta múltiples tipos de funciones:

SQL Functions

Query Language Functions: Definidas directamente en SQL

PL/pgSQL Functions

Utilizan el lenguaje procedural de PostgreSQL

Otros lenguajes

PL/Python, PL/Perl, PL/R, C, etc.

Para este aprendizaje, nos centraremos en las **SQL Functions**, que encapsulan consultas SQL en forma de función.

Componentes de una Función

Una función definida por el usuario en SQL posee varios componentes fundamentales:

01

Nombre de la función

Identificador único dentro de un esquema. Ejemplo: calcular_descuento.

03

Tipo de retorno

Define si la función devuelve un valor escalar, una fila (RETURNS TABLE) o un conjunto (SETOF). Ejemplo: RETURNS NUMERIC.

02

Parámetros de entrada

Variables que recibe la función para ejecutar la lógica. Ejemplo: (precio NUMERIC, porcentaje NUMERIC).

04

Cuerpo de la función

Bloque SQL que define la lógica. Puede ser una sola consulta o varias.

Volatilidad y Seguridad

Dos componentes adicionales importantes:

Volatilidad

Define cómo el motor optimiza la función:

- **IMMUTABLE** → siempre devuelve el mismo resultado para los mismos parámetros
- **STABLE** → puede variar entre transacciones, pero no dentro de una misma consulta
- **VOLATILE** → puede cambiar en cada ejecución (ej. funciones que dependen de NOW())

Seguridad y privilegios

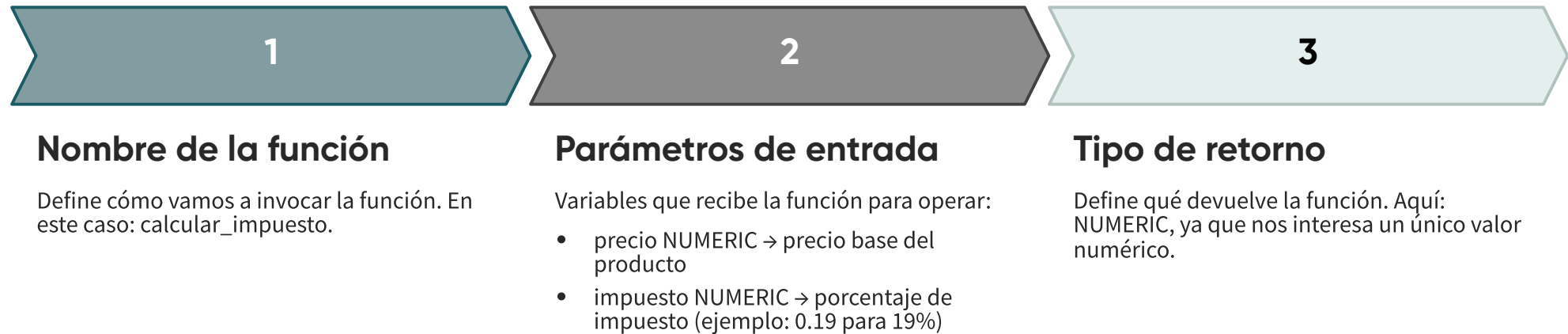
Opciones como SECURITY DEFINER permiten que la función se ejecute con privilegios del creador en lugar del usuario que la invoca.

Ejemplo Detallado: Función calcular_impuesto

Contexto

Supongamos que en una base de datos de ventas necesitamos calcular el precio final con impuesto incluido. En lugar de repetir la fórmula en múltiples queries, encapsulamos la lógica en una función.

Componentes de la Función calcular_impuesto



Más Componentes de la Función

Cuerpo de la función

Contiene la lógica en SQL que procesa los datos. Multiplicamos el precio por el impuesto y lo sumamos al precio original.

Lenguaje

Se especifica que es una función en SQL.
`LANGUAGE SQL`.

Volatilidad (opcional)

Podríamos indicar `IMMUTABLE` porque el resultado siempre será el mismo con los mismos parámetros.

Implementación en PostgreSQL

```
CREATE OR REPLACE FUNCTION calcular_impuesto(  
  precio NUMERIC,  
  impuesto NUMERIC  
)  
  RETURNS NUMERIC  
  AS $$  
  SELECT precio + (precio * impuesto);  
  $$ LANGUAGE SQL  
  IMMUTABLE;
```

Explicación de Cada Componente

Componente	Código en el ejemplo	Descripción
Nombre de la función	calcular_impuesto	Identificador único de la función.
Parámetros de entrada	(precio NUMERIC, impuesto NUMERIC)	Variables que recibe: precio base e impuesto.
Tipo de retorno	RETURNS NUMERIC	Define que devuelve un número decimal.
Cuerpo de la función	SELECT precio + (precio * impuesto);	Lógica que calcula el precio final con impuesto.
Lenguaje	LANGUAGE SQL	Indica que se usa SQL nativo.
Volatilidad	IMMUTABLE	El resultado siempre es igual con los mismos parámetros.

Uso de la Función

```
SELECT calcular_impuesto(100, 0.19);
```

Resultado esperado:

119.00



Caso Aplicado en Tabla de Ventas

Supongamos la tabla:

```
CREATE TABLE ventas (  
  id SERIAL PRIMARY KEY,  
  producto TEXT,  
  precio NUMERIC,  
  impuesto NUMERIC  
);
```

Insertamos algunos datos:

```
INSERT INTO ventas (producto, precio, impuesto) VALUES  
( 'Laptop', 1200, 0.19),  
( 'Teléfono', 800, 0.19),  
( 'Tablet', 500, 0.10);
```

Aplicando la Función sobre la Tabla

Ahora aplicamos la función sobre la tabla:

```
SELECT producto,  
precio,  
calcular_impuesto(precio, impuesto) AS precio_final  
FROM ventas;
```

Salida esperada:

producto	precio	precio_final
Laptop	1200	1428.00
Teléfono	800	952.00
Tablet	500	550.00

Reflexión Didáctica

Este ejemplo permite al estudiante identificar visualmente cada componente de una función y comprender:



Que una función no es solo código SQL, sino una estructura con partes definidas.



Que su uso en consultas simplifica la lógica y hace más mantenible el sistema.



Que puede aplicarse tanto a operaciones individuales como a consultas sobre tablas completas.

Casos de Uso de una Función

Las funciones definidas por el usuario son aplicables en múltiples escenarios:



Cálculos recurrentes

Ejemplo: calcular IVA, descuentos o comisiones.

```
SELECT calcular_descuento(100, 0.2);  
-- Devuelve: 80
```



Transformación de datos

Funciones que formatean nombres, concatenan campos o normalizan datos.



Validación de reglas de negocio

Ejemplo: verificar si un cliente es elegible para un crédito.

Más Casos de Uso

Consultas complejas reutilizables

En lugar de reescribir joins y subconsultas, encapsularlos en una función.

Agregaciones personalizadas

Funciones que complementan las agregaciones estándar (SUM, AVG, etc.) con lógica propia.

Interoperabilidad con aplicaciones

Muchas veces, aplicaciones externas llaman funciones en lugar de queries crudas, para seguridad y simplicidad.

Implementación con Query Language Functions

En PostgreSQL, la sintaxis básica para crear una función SQL es:

```
CREATE FUNCTION nombre_funcion(parametros)
RETURNS tipo_retorno
AS $$
-- cuerpo de la función
SELECT ...
$$ LANGUAGE SQL;
```

Ejemplo simple: calcular el precio final con descuento

```
CREATE FUNCTION calcular_descuento(precio NUMERIC, porcentaje NUMERIC)
RETURNS NUMERIC
AS $$
SELECT precio - (precio * porcentaje);
$$ LANGUAGE SQL;
```

Uso:

```
SELECT calcular_descuento(200, 0.15);
-- Resultado: 170
```

Ejemplo Avanzado: Retornar una Tabla

```
CREATE FUNCTION empleados_departamento(dep_id INT)
RETURNS TABLE(nombre TEXT, salario NUMERIC)
AS $$
SELECT e.nombre, e.salario
FROM empleados e
WHERE e.departamento_id = dep_id;
$$ LANGUAGE SQL;
```

Uso:

```
SELECT * FROM empleados_departamento(2);
```

Esto devuelve todos los empleados de un departamento, encapsulando la lógica en una sola llamada.

Parte Práctica: Sistema de Ventas

Supongamos que tenemos la siguiente tabla:

```
CREATE TABLE ventas (  
  id SERIAL PRIMARY KEY,  
  cliente TEXT,  
  producto TEXT,  
  cantidad INT,  
  precio_unitario NUMERIC,  
  fecha DATE  
);
```

Ejercicio 1: Función para calcular el total de una venta

```
CREATE FUNCTION total_venta(cantidad INT, precio NUMERIC)  
RETURNS NUMERIC  
AS $$  
  SELECT cantidad * precio;  
$$ LANGUAGE SQL;
```

Prueba:

```
SELECT total_venta(5, 120);  
-- Resultado: 600
```

Ejercicios Adicionales y Cierre

Ejercicio 2: Función para calcular ingresos de un cliente

```
CREATE FUNCTION ingresos_cliente(nombre_cliente TEXT)
RETURNS NUMERIC
AS $$
SELECT SUM(cantidad * precio_unitario)
FROM ventas
WHERE cliente = nombre_cliente;
$$ LANGUAGE SQL;
```

Ejercicio 3: Función para obtener ventas en un rango de fechas

```
CREATE FUNCTION ventas_rango(fecha_inicio DATE, fecha_fin DATE)
RETURNS TABLE(cliente TEXT, total NUMERIC)
AS $$
SELECT cliente, SUM(cantidad * precio_unitario) AS total
FROM ventas
WHERE fecha BETWEEN fecha_inicio AND fecha_fin
GROUP BY cliente;
$$ LANGUAGE SQL;
```

Actividad integradora: Se puede proponer como ejercicio integrador que los estudiantes construyan una función que calcule el producto más vendido en un rango de fechas y lo retorne junto con el total de ingresos generados. Esto los obliga a definir parámetros de entrada, usar RETURNS TABLE, y aplicar GROUP BY y ORDER BY dentro de la función.